

Snickr Collaboration System

Project 2 Web Interface Report

CS 6083 Database Systems Project
Shuhan Cai sc12227
Yang Zhang yz12043

April 30, 2026

1 Overview

The second phase of the snickr project implements a browser-based interface on top of the relational database designed in Project 1. The system is a Slack-like collaboration site with users, workspaces, channels, invitations, and messages. The web application is implemented in Python using Flask for HTTP request handling and sessions, and PostgreSQL remains the persistent backend. The Project 1 schema was kept intentionally close to the original design so the implementation could use clear SQL joins for authorization and browsing.

The completed interface supports registration, login, workspace creation, channel creation, workspace invitations, channel invitations, invitation responses, message posting, channel browsing, and keyword search over messages that the logged-in user is allowed to read. The site runs locally for the demo, but it uses normal web application conventions: all operations go through a standard browser, session state is stored in a secure Flask session cookie, and database operations use parameterized SQL.

2 Project 1 Design Revisions

The Project 1 design already separated workspace membership from channel membership. This decision became important in Project 2 because it lets the application enforce Slack-like channel access rules. A workspace member can discover and open public channels in that workspace; opening a public channel adds the user to `channel_members`. Private and direct channels are shown only to users who already have channel membership. The search route follows the same rule: public channel messages are searchable by workspace members, while private and direct messages require channel membership.

The main Project 2 revision is operational rather than structural. The web application treats multi-step actions as transactions. Creating a workspace inserts the workspace and immediately inserts the creator as an administrator. Creating a channel inserts the channel and immediately inserts the creator into `channel_members`. Accepting an invitation updates the invitation row and inserts the corresponding membership row in one database transaction. Creating a direct channel inserts both original participants into `channel_members` in the same transaction. Withdrawing a message is implemented as a soft update: the message row remains in the database, but `withdrawn_at` and `withdrawn_by` record the withdrawal.

Password handling was also improved for new web users. Project 1 used `password_hash` as an attribute but the sample data contains readable placeholder values for demo convenience. The

Project 2 registration route stores Werkzeug password hashes. The login route still accepts the legacy sample placeholders such as `hash_alice` so the provided sample data can be demonstrated without editing all rows.

3 Architecture

The application has three layers. The browser renders HTML pages and submits forms. Flask receives the requests, validates the session, performs the application-level authorization checks, and sends parameterized SQL statements to PostgreSQL. PostgreSQL enforces primary keys, foreign keys, uniqueness, check constraints, and transactional consistency.

Component	Responsibility
<code>app.py</code>	Flask routes, session loading, form handling, access checks, and parameterized SQL calls.
<code>schema.sql</code>	PostgreSQL table definitions, keys, foreign keys, checks, and indexes.
<code>sample_data.sql</code>	Demo users, workspaces, invitations, channels, memberships, and messages.
<code>templates/</code>	Server-rendered HTML pages. Jinja autoescaping protects dynamic output by default.
<code>static/style.css</code>	Responsive visual design for the browser demo.

The database connection string is read from the `DATABASE_URL` environment variable, with local defaults supplied by `.env` or the fallback URL in `app.py`. The command `flask --app app init-db` loads `schema.sql` and `sample_data.sql`; `flask --app app init-db --reset` rebuilds an existing demo database.

4 User Sessions

A successful login stores the user's `user_id` in the Flask session. Before each request, the application loads the current user's public identity from PostgreSQL into `g.user`. Routes that require authentication are protected by the `login_required` decorator. If no user is logged in, the request is redirected to the login page.

This design keeps database accounts separate from application users. The web server connects to PostgreSQL with one configured database account, and each browser session maps to a row in the `users` table. This matches the assumption documented in Project 1 and is easier to demo than creating a separate PostgreSQL role for each browser user.

5 Implemented User Features

5.1 Registration and Login

New users register with email, username, nickname, and password. The `users` table rejects duplicate emails and usernames. New passwords are stored using `generate_password_hash`. Login accepts either username or email. Existing sample accounts can log in with their placeholder sample password values, for example username `alice` and password `hash_alice`.

5.2 Workspace Management

The dashboard lists all workspaces where the logged-in user is a member. A user can create a new workspace from the dashboard. The create operation is transactional: first the row is inserted into `workspaces`, then the creator is inserted into `workspace_members` with role `admin`. Workspace administrators can invite other users by email.

5.3 Channel Management

Inside a workspace, the user sees all public channels in that workspace and any private or direct channels where the user is already a channel member. Public channels are joinable by opening them, which inserts the current user into `channel_members`. The user can create a public, private, or direct channel. Channel creation inserts the new `channels` row and inserts the creator into `channel_members`. For public and private channels, the creation form can also record channel invitations for a comma-separated list of usernames. The application first verifies that every invited username belongs to the current workspace, so a channel invitation is not created for a user who cannot accept it. Direct channels require exactly one other workspace member; that member is added directly to `channel_members`, and direct channels do not support later invitations.

5.4 Invitations

The dashboard shows pending workspace invitations and valid channel invitations. Channel invitations are displayed only when the invited user is already a member of the channel's workspace. Accepting a workspace invitation updates the invitation status and inserts the user into `workspace_members`. Accepting a channel invitation updates the invitation status and inserts the user into `channel_members`; the application first checks that the user is a workspace member. Declining an invitation updates only the invitation status and response timestamp. The dashboard also polls `/dashboard/invitations.json` every three seconds so new workspace and channel invitations appear without refreshing the whole page.

5.5 Messaging, Browsing, and Bookmarks

The channel page lists messages in chronological order and allows channel members to post new messages. The database rejects blank message bodies with a check constraint, and the web route also strips input and rejects empty posts before sending SQL. Bookmarkable URLs identify workspaces and channels: `/workspaces/<id>` and `/channels/<id>`.

To make channel pages update without interrupting the composer, the browser uses local JSON polling. Every three seconds, the channel page requests `/channels/<id>/messages.json?after=<last_message_id>`. The server returns only messages with larger ids, and the JavaScript appends them to the message list without reloading the page or touching the text area where the user may be typing. The same endpoint also returns withdrawn message ids, so another browser window can replace an already displayed message with the withdrawn placeholder without a full refresh. Hidden browser tabs skip polling until they are visible again.

Users can withdraw their own messages. A withdrawal does not delete the row; instead the application records the withdrawal timestamp and user, then renders a neutral placeholder in the channel timeline. Other users cannot withdraw a message they did not send. Withdrawn messages are excluded from keyword search results. If a withdrawn message had been bookmarked, the withdrawal transaction also removes the corresponding message bookmark entries so dashboards do not link to withdrawn content.

The interface also implements in-application bookmarks. A logged-in user can bookmark a workspace, a channel, an individual message, or a search result page. Bookmarks are stored in the `bookmarks` table with the owning user, a label, a target type, and the relative target URL. Message bookmarks use channel anchors such as `/channels/1#message-42`, so the dashboard link opens the channel at the saved message. The dashboard lists the current user's bookmarks so the user can reopen saved workspaces, channels, messages, and searches with one click. Bookmark targets are restricted to internal workspace, channel, message-anchor, and search URLs.

5.6 Search

The search page accepts a keyword and returns only matching messages that are accessible to the logged-in user. The query always checks `workspace_members`. It also checks `channel_members` for private and direct channels; public channel messages are accessible to all members of the containing workspace. This reproduces the Project 1 test case for the word `perpendicular`: Cara can see matching public messages in workspaces where she is a member, but not the matching private message in `#hiring` because she is not a member of that channel. Search uses a GET query parameter, for example `/search?q=perpendicular`, so search results can be bookmarked both by the browser and by the application's own bookmark feature.

6 SQL Injection Protection

All application SQL uses `psycopg2` parameter binding with `%s` placeholders. User inputs such as usernames, email addresses, channel names, message bodies, invitation ids, and search keywords are passed as SQL parameters rather than concatenated into SQL strings. This is the main defense against SQL injection.

For example, the search route uses:

```
WHERE m.body ILIKE '%%' || %s || '%%'
AND (c.channel_type = 'public' OR cm.user_id IS NOT NULL)
```

The keyword value is sent separately from the SQL text, so a malicious keyword is treated as data, not executable SQL. Similarly, all inserts and updates use bound parameters.

7 Cross-Site Scripting Protection

The interface uses Jinja templates. Jinja autoescapes dynamic HTML output by default for `.html` templates. Message bodies, nicknames, workspace names, and channel names are therefore escaped before being returned to the browser. The application does not mark user-supplied text as safe HTML. This means a message containing a string such as `<script>` will display as text rather than executing in another user's browser.

8 Concurrency and Transactions

Several application actions affect more than one table. These are wrapped in database transactions using the `psycopg2` connection context manager. If any statement in the block fails, the transaction is rolled back. This prevents partial state such as a workspace without its creator membership or an accepted invitation without the corresponding membership.

The database also uses uniqueness constraints to handle concurrent requests. For example, duplicate channel invitations are prevented by the unique constraint on `(channel_id, invited_user_id)`, duplicate channel memberships are prevented by the primary key on `(channel_id, user_id)`, and duplicate channel names inside one workspace are prevented by `UNIQUE (workspace_id, name)`. The application uses `ON CONFLICT DO NOTHING` for invitation inserts and membership inserts where repeated clicks should be harmless, such as accepting an invitation or opening a public channel concurrently in two browser tabs.

9 Demo Data

The sample database contains twelve users, four workspaces, twelve channels, workspace invitations, channel invitations, and forty messages. The data was chosen to demonstrate public channels, private channels, direct channels, accepted invitations, pending invitations, declined invitations, revoked invitations, administrators, normal members, search access control, and user bookmarks and message withdrawals created during the demo.

Entity or relationship	Rows
Users	12
Workspaces	4
Workspace memberships	17
Workspace invitations	8
Channels	12
Channel memberships	31
Channel invitations	12
Messages	40
User bookmarks	Created interactively

10 How to Run the System

Install dependencies:

```
python -m pip install -r requirements.txt
```

Create a PostgreSQL database named `snickr`. If needed, set the database URL:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/snickr
```

On Windows PowerShell, use:

```
$env:DATABASE_URL="postgresql://postgres:postgres@localhost:5432/snickr"
```

Initialize the schema and sample data:

```
flask --app app init-db
```

Run the development server:

```
flask --app app run --debug
```

Then open `http://127.0.0.1:5000` in a standard browser.

11 Demo Script

A useful demo starts by logging in as `alice` with password `hash_alice`. Alice can open Acme Lab, create a public channel, post a message, invite another user to the workspace, invite a workspace member to a private or public channel, and create a direct channel with exactly one other workspace member. Alice can bookmark Acme Lab, a channel, and an individual message; these saved links appear in the dashboard bookmark list. Alice can also withdraw one of her own messages to show the soft-delete behavior in the channel timeline. Next, log out and log in as the invited user to show that pending invitations appear on the dashboard and can be accepted or declined. Finally, log in as `cara`, search for `perpendicular`, and bookmark the search result page. The result list shows public messages in Cara’s workspaces and hides private, direct, or withdrawn messages that Cara should not read.

12 Source Code Appendix

12.1 `schema.sql`

```
DROP TABLE IF EXISTS messages CASCADE;
DROP TABLE IF EXISTS bookmarks CASCADE;
DROP TABLE IF EXISTS channel_invitations CASCADE;
DROP TABLE IF EXISTS channel_members CASCADE;
DROP TABLE IF EXISTS channels CASCADE;
DROP TABLE IF EXISTS workspace_invitations CASCADE;
DROP TABLE IF EXISTS workspace_members CASCADE;
DROP TABLE IF EXISTS workspaces CASCADE;
DROP TABLE IF EXISTS users CASCADE;

CREATE TABLE users (
  user_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  username VARCHAR(80) NOT NULL UNIQUE,
  nickname VARCHAR(120) NOT NULL,
  password_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT users_email_format_ck CHECK (position('@' in email) > 1)
);

CREATE TABLE workspaces (
  workspace_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name VARCHAR(120) NOT NULL UNIQUE,
  description TEXT,
  created_by BIGINT NOT NULL REFERENCES users(user_id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspace_members (
  workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id) ON DELETE CASCADE,
  user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  role VARCHAR(20) NOT NULL DEFAULT 'member',
  joined_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (workspace_id, user_id),
  CONSTRAINT workspace_members_role_ck CHECK (role IN ('admin', 'member'))
);
```

```

CREATE TABLE workspace_invitations (
    workspace_invitation_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id) ON DELETE CASCADE
,
    invited_email VARCHAR(255) NOT NULL,
    invited_user_id BIGINT REFERENCES users(user_id),
    invited_by BIGINT NOT NULL REFERENCES users(user_id),
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    invited_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    responded_at TIMESTAMPTZ,
    UNIQUE (workspace_id, invited_email),
    CONSTRAINT workspace_invitations_status_ck
        CHECK (status IN ('pending', 'accepted', 'declined', 'revoked'))
);

CREATE TABLE bookmarks (
    bookmark_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    label VARCHAR(160) NOT NULL,
    target_type VARCHAR(20) NOT NULL,
    target_url TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (user_id, target_url),
    CONSTRAINT bookmarks_type_ck
        CHECK (target_type IN ('workspace', 'channel', 'search', 'message')),
    CONSTRAINT bookmarks_url_not_blank_ck CHECK (length(btrim(target_url)) > 0)
);

CREATE TABLE channels (
    channel_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id) ON DELETE CASCADE
,
    name VARCHAR(120) NOT NULL,
    channel_type VARCHAR(20) NOT NULL,
    created_by BIGINT NOT NULL REFERENCES users(user_id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (workspace_id, name),
    CONSTRAINT channels_type_ck CHECK (channel_type IN ('public', 'private', 'direct'))
);

CREATE TABLE channel_members (
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id) ON DELETE CASCADE,
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    joined_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (channel_id, user_id)
);

CREATE TABLE channel_invitations (
    channel_invitation_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id) ON DELETE CASCADE,
    invited_user_id BIGINT NOT NULL REFERENCES users(user_id),
    invited_by BIGINT NOT NULL REFERENCES users(user_id),
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    invited_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    responded_at TIMESTAMPTZ,
    UNIQUE (channel_id, invited_user_id),
    CONSTRAINT channel_invitations_status_ck
        CHECK (status IN ('pending', 'accepted', 'declined', 'revoked'))
);

```

```

);

CREATE TABLE messages (
    message_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id) ON DELETE CASCADE,
    sender_id BIGINT NOT NULL REFERENCES users(user_id),
    body TEXT NOT NULL,
    posted_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    withdrawn_at TIMESTAMPTZ,
    withdrawn_by BIGINT REFERENCES users(user_id),
    CONSTRAINT messages_body_not_blank_ck CHECK (length(btrim(body)) > 0)
);

CREATE INDEX idx_messages_channel_time ON messages(channel_id, posted_at, message_id);
CREATE INDEX idx_messages_sender_time ON messages(sender_id, posted_at, message_id);
CREATE INDEX idx_channel_invitations_pending ON channel_invitations(channel_id, status
, invited_at);
CREATE INDEX idx_bookmarks_user_time ON bookmarks(user_id, created_at DESC,
bookmark_id DESC);

-- Optional performance optimization for substring keyword search:
-- CREATE EXTENSION IF NOT EXISTS pg_trgm;
-- CREATE INDEX idx_messages_body_trgm_like ON messages USING gin (body gin_trgm_ops);

```

12.2 sample_data.sql

```

INSERT INTO users(user_id, email, username, nickname, password_hash, created_at)
VALUES
(1, 'alice@example.com', 'alice', 'Alice A.', 'hash_alice', '2026-04-01
09:00:00-04'),
(2, 'bob@example.com', 'bob', 'Bobby', 'hash_bob', '2026-04-01 09:05:00-04'),
(3, 'cara@example.com', 'cara', 'Cara', 'hash_cara', '2026-04-01 09:10:00-04'),
(4, 'dan@example.com', 'dan', 'Dan', 'hash_dan', '2026-04-01 09:15:00-04'),
(5, 'erin@example.com', 'erin', 'Erin', 'hash_erin', '2026-04-01 09:20:00-04'),
(6, 'maya@example.com', 'maya', 'Maya', 'hash_maya', '2026-04-01 09:25:00-04'),
(7, 'gina@example.com', 'gina', 'Gina', 'hash_gina', '2026-04-01 09:30:00-04'),
(8, 'hank@example.com', 'hank', 'Hank', 'hash_hank', '2026-04-01 09:35:00-04'),
(9, 'ivy@example.com', 'ivy', 'Ivy', 'hash_ivy', '2026-04-01 09:40:00-04'),
(10, 'jack@example.com', 'jack', 'Jack', 'hash_jack', '2026-04-01 09:45:00-04'),
(11, 'ken@example.com', 'ken', 'Ken', 'hash_ken', '2026-04-01 09:50:00-04'),
(12, 'lara@example.com', 'lara', 'Lara', 'hash_lara', '2026-04-01 09:55:00-04');

INSERT INTO workspaces(workspace_id, name, description, created_by, created_at) VALUES
(1, 'Acme Lab', 'Engineering collaboration for Acme Lab.', 1, '2026-04-02
08:00:00-04'),
(2, 'Coop Board', 'Building coop board workspace.', 3, '2026-04-03 08:00:00-04'),
(3, 'Design Guild', 'Product design and research workspace.', 6, '2026-04-04
08:00:00-04'),
(4, 'Study Hub', 'Course planning and exam preparation workspace.', 9, '2026-04-05
08:00:00-04');

INSERT INTO workspace_members(workspace_id, user_id, role, joined_at) VALUES
(1, 1, 'admin', '2026-04-02 08:00:00-04'),
(1, 2, 'admin', '2026-04-02 10:00:00-04'),
(1, 3, 'member', '2026-04-03 12:00:00-04'),
(1, 6, 'member', '2026-04-08 09:00:00-04'),
(1, 7, 'member', '2026-04-09 09:00:00-04'),

```



```
(1, 8, 'member', '2026-04-10 09:00:00-04'),
(2, 3, 'admin', '2026-04-03 08:00:00-04'),
(2, 1, 'member', '2026-04-04 12:00:00-04'),
(2, 5, 'member', '2026-04-05 12:00:00-04'),
(2, 11, 'member', '2026-04-06 12:00:00-04'),
(3, 6, 'admin', '2026-04-04 08:00:00-04'),
(3, 7, 'member', '2026-04-04 10:00:00-04'),
(3, 8, 'admin', '2026-04-04 11:00:00-04'),
(3, 10, 'member', '2026-04-05 11:00:00-04'),
(4, 9, 'admin', '2026-04-05 08:00:00-04'),
(4, 10, 'member', '2026-04-05 10:00:00-04'),
(4, 12, 'member', '2026-04-06 10:00:00-04');
```

```
INSERT INTO workspace_invitations(workspace_invitation_id, workspace_id, invited_email
, invited_user_id, invited_by, status, invited_at, responded_at) VALUES
(1, 1, 'dan@example.com', 4, 1, 'pending', '2026-04-15 09:00:00-04', NULL),
(2, 1, 'cara@example.com', 3, 1, 'accepted', '2026-04-02 09:00:00-04', '2026-04-03
12:00:00-04'),
(3, 2, 'alice@example.com', 1, 3, 'accepted', '2026-04-03 09:00:00-04', '
2026-04-04 12:00:00-04'),
(4, 1, 'erin@example.com', 5, 2, 'pending', '2026-04-20 09:00:00-04', NULL),
(5, 3, 'dan@example.com', 4, 6, 'declined', '2026-04-07 09:00:00-04', '2026-04-08
16:30:00-04'),
(6, 4, 'alice@example.com', 1, 9, 'pending', '2026-04-18 14:00:00-04', NULL),
(7, 3, 'jack@example.com', 10, 6, 'accepted', '2026-04-04 09:00:00-04', '
2026-04-05 11:00:00-04'),
(8, 2, 'maya@example.com', 6, 3, 'revoked', '2026-04-10 09:30:00-04', '2026-04-12
17:00:00-04');
```

```
INSERT INTO channels(channel_id, workspace_id, name, channel_type, created_by,
created_at) VALUES
(1, 1, 'general', 'public', 1, '2026-04-02 08:15:00-04'),
(2, 1, 'product', 'public', 2, '2026-04-05 10:00:00-04'),
(3, 1, 'hiring', 'private', 1, '2026-04-06 11:00:00-04'),
(4, 1, 'dm-alice-bob', 'direct', 1, '2026-04-07 12:00:00-04'),
(5, 2, 'maintenance', 'public', 3, '2026-04-04 09:00:00-04'),
(6, 1, 'sprint-planning', 'public', 6, '2026-04-08 09:15:00-04'),
(7, 1, 'leadership', 'private', 1, '2026-04-09 10:00:00-04'),
(8, 1, 'dm-cara-maya', 'direct', 3, '2026-04-10 11:00:00-04'),
(9, 3, 'design-general', 'public', 6, '2026-04-04 08:30:00-04'),
(10, 3, 'ux-research', 'private', 8, '2026-04-05 13:00:00-04'),
(11, 4, 'study-general', 'public', 9, '2026-04-05 08:30:00-04'),
(12, 4, 'exam-prep', 'private', 9, '2026-04-06 15:00:00-04');
```

```
INSERT INTO channel_members(channel_id, user_id, joined_at) VALUES
(1, 1, '2026-04-02 08:15:00-04'),
(1, 2, '2026-04-02 10:00:00-04'),
(1, 3, '2026-04-03 12:05:00-04'),
(2, 1, '2026-04-05 10:00:00-04'),
(2, 2, '2026-04-05 10:01:00-04'),
(3, 1, '2026-04-06 11:00:00-04'),
(3, 2, '2026-04-06 11:05:00-04'),
(4, 1, '2026-04-07 12:00:00-04'),
(4, 2, '2026-04-07 12:00:00-04'),
(5, 3, '2026-04-04 09:00:00-04'),
(5, 1, '2026-04-04 12:00:00-04'),
(6, 1, '2026-04-08 09:15:00-04'),
(6, 2, '2026-04-08 09:16:00-04'),
(6, 3, '2026-04-08 09:17:00-04'),
```

```
(6, 6, '2026-04-08 09:18:00-04'),
(6, 7, '2026-04-09 09:10:00-04'),
(7, 1, '2026-04-09 10:00:00-04'),
(7, 2, '2026-04-09 10:05:00-04'),
(8, 3, '2026-04-10 11:00:00-04'),
(8, 6, '2026-04-10 11:00:00-04'),
(9, 6, '2026-04-04 08:30:00-04'),
(9, 7, '2026-04-04 10:05:00-04'),
(9, 8, '2026-04-04 11:05:00-04'),
(9, 10, '2026-04-05 11:05:00-04'),
(10, 6, '2026-04-05 13:00:00-04'),
(10, 8, '2026-04-05 13:05:00-04'),
(11, 9, '2026-04-05 08:30:00-04'),
(11, 10, '2026-04-05 10:05:00-04'),
(11, 12, '2026-04-06 10:05:00-04'),
(12, 9, '2026-04-06 15:00:00-04'),
(12, 12, '2026-04-06 15:05:00-04');
```

```
INSERT INTO channel_invitations(channel_invitation_id, channel_id, invited_user_id,
invited_by, status, invited_at, responded_at) VALUES
(1, 1, 4, 1, 'pending', '2026-04-15 09:00:00-04', NULL),
(2, 2, 3, 2, 'pending', '2026-04-17 09:00:00-04', NULL),
(3, 2, 4, 2, 'pending', '2026-04-24 09:00:00-04', NULL),
(4, 3, 3, 1, 'pending', '2026-04-16 09:00:00-04', NULL),
(5, 5, 1, 3, 'accepted', '2026-04-03 09:00:00-04', '2026-04-04 12:00:00-04'),
(6, 6, 4, 6, 'pending', '2026-04-18 09:00:00-04', NULL),
(7, 6, 5, 6, 'pending', '2026-04-23 09:00:00-04', NULL),
(8, 7, 3, 1, 'pending', '2026-04-19 09:00:00-04', NULL),
(9, 9, 10, 6, 'accepted', '2026-04-04 09:00:00-04', '2026-04-05 11:00:00-04'),
(10, 10, 7, 8, 'declined', '2026-04-06 09:00:00-04', '2026-04-07 10:00:00-04'),
(11, 11, 11, 9, 'pending', '2026-04-20 09:00:00-04', NULL),
(12, 12, 10, 9, 'revoked', '2026-04-07 09:00:00-04', '2026-04-08 12:00:00-04');
```

```
INSERT INTO messages(message_id, channel_id, sender_id, body, posted_at) VALUES
(1, 1, 1, 'Welcome to Acme Lab general.', '2026-04-02 08:20:00-04'),
(2, 1, 2, 'The perpendicular test rig is ready.', '2026-04-03 10:00:00-04'),
(3, 1, 3, 'I can review the onboarding checklist.', '2026-04-03 12:30:00-04'),
(4, 2, 2, 'Product roadmap review at 3pm.', '2026-04-05 10:10:00-04'),
(5, 3, 1, 'Private hiring notes mention perpendicular leadership fit.', '
2026-04-06 12:00:00-04'),
(6, 4, 1, 'Bob, can you check the direct channel setup?', '2026-04-07 12:05:00-04'
),
(7, 5, 3, 'Maintenance elevator request is open.', '2026-04-04 09:30:00-04'),
(8, 5, 1, 'The stair railing is perpendicular to the landing.', '2026-04-04
12:15:00-04'),
(9, 2, 1, 'Launch checklist moved to the product board.', '2026-04-08 09:20:00-04'
),
(10, 2, 1, 'Please review the signup copy before Friday.', '2026-04-08 10:05:00-04
'),
(11, 2, 1, 'Roadmap notes now include the browser demo path.', '2026-04-08
11:30:00-04'),
(12, 6, 6, 'Sprint planning starts with the database migration review.', '
2026-04-08 09:30:00-04'),
(13, 6, 3, 'I will bring the invitation edge cases.', '2026-04-08 09:45:00-04'),
(14, 6, 7, 'The QA checklist needs one more search scenario.', '2026-04-09
09:40:00-04'),
(15, 7, 1, 'Leadership notes: keep admin controls simple.', '2026-04-09
10:15:00-04'),
```

```

(16, 7, 1, 'Only channel members should see this budget thread.', '2026-04-09
10:45:00-04'),
(17, 8, 3, 'Maya, can you help with the onboarding screen?', '2026-04-10
11:10:00-04'),
(18, 8, 6, 'Yes, I will mock up the workspace selector.', '2026-04-10 11:12:00-04'
),
(19, 5, 3, 'The elevator vendor confirmed Tuesday morning.', '2026-04-05
09:15:00-04'),
(20, 5, 1, 'I added the maintenance note to the shared log.', '2026-04-05
12:30:00-04'),
(21, 5, 3, 'We should invite Ken to the next board update.', '2026-04-06
09:10:00-04'),
(22, 9, 6, 'Welcome to the design guild workspace.', '2026-04-04 08:45:00-04'),
(23, 9, 7, 'I posted color contrast examples for the mockup.', '2026-04-04
10:20:00-04'),
(24, 9, 8, 'Research notes are linked from the channel topic.', '2026-04-04
11:30:00-04'),
(25, 9, 10, 'I can test the responsive layout tomorrow.', '2026-04-05 11:30:00-04'
),
(26, 10, 6, 'Interview script draft is ready for review.', '2026-04-05 13:15:00-04
'),
(27, 10, 8, 'Private research findings should stay in this channel.', '2026-04-05
13:45:00-04'),
(28, 11, 9, 'Study Hub is open for database review.', '2026-04-05 08:45:00-04'),
(29, 11, 10, 'I uploaded the normalization notes.', '2026-04-05 10:30:00-04'),
(30, 11, 12, 'Can we schedule a transaction isolation review?', '2026-04-06
10:30:00-04'),
(31, 12, 9, 'Exam prep channel starts with query optimization.', '2026-04-06
15:20:00-04'),
(32, 12, 12, 'I will summarize indexes and explain plans.', '2026-04-06
15:45:00-04'),
(33, 6, 1, 'The demo account list should include invited users.', '2026-04-10
09:00:00-04'),
(34, 2, 1, 'Search examples should include roadmap and browser.', '2026-04-10
10:00:00-04'),
(35, 9, 6, 'The dashboard should show unread channel counts.', '2026-04-10
11:00:00-04'),
(36, 10, 8, 'User quotes need anonymized labels.', '2026-04-10 13:00:00-04'),
(37, 11, 9, 'Bookmarkable workspace URLs would help during the demo.', '2026-04-10
14:00:00-04'),
(38, 12, 12, 'Prepared statements are on my study checklist.', '2026-04-10
15:00:00-04'),
(39, 4, 1, 'This direct message verifies one-to-one browsing.', '2026-04-11
12:00:00-04'),
(40, 8, 3, 'Direct messages should appear only to both participants.', '2026-04-11
12:30:00-04');

SELECT setval(pg_get_serial_sequence('users', 'user_id'), (SELECT max(user_id) FROM
users));
SELECT setval(pg_get_serial_sequence('workspaces', 'workspace_id'), (SELECT max(
workspace_id) FROM workspaces));
SELECT setval(pg_get_serial_sequence('workspace_invitations', 'workspace_invitation_id
'), (SELECT max(workspace_invitation_id) FROM workspace_invitations));
SELECT setval(pg_get_serial_sequence('channels', 'channel_id'), (SELECT max(channel_id
) FROM channels));
SELECT setval(pg_get_serial_sequence('channel_invitations', 'channel_invitation_id'),
(SELECT max(channel_invitation_id) FROM channel_invitations));
SELECT setval(pg_get_serial_sequence('messages', 'message_id'), (SELECT max(message_id
) FROM messages));

```

12.3 app.py

```
import os
from functools import wraps
from urllib.parse import urlparse

import click
import psycopg2
import psycopg2.extras
from flask import (
    Flask,
    flash,
    g,
    jsonify,
    redirect,
    render_template,
    request,
    session,
    url_for,
)
from werkzeug.security import check_password_hash, generate_password_hash

app = Flask(__name__)

def load_env_file(filename=".env"):
    if not os.path.exists(filename):
        return
    with open(filename, encoding="utf-8") as env_file:
        for line in env_file:
            line = line.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            key, value = line.split("=", 1)
            os.environ.setdefault(key.strip(), value.strip().strip("'"))

load_env_file()

app.config["SECRET_KEY"] = os.environ.get("SECRET_KEY", "dev-only-change-me")
app.config["DATABASE_URL"] = os.environ.get(
    "DATABASE_URL", "postgresql://postgres:csh1q2w3e4r@localhost:5432/snicker"
)

def get_db():
    if "db" not in g:
        g.db = psycopg2.connect(
            app.config["DATABASE_URL"],
            cursor_factory=psycopg2.extras.RealDictCursor,
        )
    return g.db
```

```

@app.teardown_appcontext
def close_db(_error=None):
    db = g.pop("db", None)
    if db is not None:
        db.close()

def query_all(sql, params=()):
    with get_db().cursor() as cur:
        cur.execute(sql, params)
        return cur.fetchall()

def query_one(sql, params=()):
    with get_db().cursor() as cur:
        cur.execute(sql, params)
        return cur.fetchone()

def ensure_bookmarks_schema():
    if app.config.get("BOOKMARKS_SCHEMA_READY"):
        return
    with get_db(), get_db().cursor() as cur:
        cur.execute(
            """
            CREATE TABLE IF NOT EXISTS bookmarks (
                bookmark_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
                user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
                label VARCHAR(160) NOT NULL,
                target_type VARCHAR(20) NOT NULL,
                target_url TEXT NOT NULL,
                created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
                UNIQUE (user_id, target_url),
                CONSTRAINT bookmarks_type_ck
                    CHECK (target_type IN ('workspace', 'channel', 'search', 'message
'')),
                CONSTRAINT bookmarks_url_not_blank_ck
                    CHECK (length(btrim(target_url)) > 0)
            )
            """
        )
        cur.execute("ALTER TABLE bookmarks DROP CONSTRAINT IF EXISTS bookmarks_type_ck")
    cur.execute(
        """
        ALTER TABLE bookmarks
        ADD CONSTRAINT bookmarks_type_ck
        CHECK (target_type IN ('workspace', 'channel', 'search', 'message'))
        """
    )
    cur.execute(
        """
        CREATE INDEX IF NOT EXISTS idx_bookmarks_user_time
        ON bookmarks(user_id, created_at DESC, bookmark_id DESC)
        """
    )
    app.config["BOOKMARKS_SCHEMA_READY"] = True

```

```

def ensure_message_withdrawal_schema():
    if app.config.get("MESSAGE_WITHDRAWAL_SCHEMA_READY"):
        return
    with get_db(), get_db().cursor() as cur:
        cur.execute("ALTER TABLE messages ADD COLUMN IF NOT EXISTS withdrawn_at
TIMESTAMPZ")
        cur.execute(
            """
            ALTER TABLE messages
            ADD COLUMN IF NOT EXISTS withdrawn_by BIGINT REFERENCES users(user_id)
            """
        )
    app.config["MESSAGE_WITHDRAWAL_SCHEMA_READY"] = True

def normalize_bookmark_target(target_url):
    parsed = urlparse(target_url.strip())
    if parsed.scheme or parsed.netloc:
        return None
    if parsed.path.startswith("/workspaces/"):
        return "workspace", parsed.path
    if parsed.path.startswith("/channels/"):
        if parsed.fragment.startswith("message-"):
            return "message", f"{parsed.path}#{parsed.fragment}"
        return "channel", parsed.path
    if parsed.path == "/search":
        query = f"?{parsed.query}" if parsed.query else ""
        return "search", parsed.path + query
    return None

def bookmark_for(target_url):
    ensure_bookmarks_schema()
    return query_one(
        """
        SELECT bookmark_id
        FROM bookmarks
        WHERE user_id = %s AND target_url = %s
        """,
        (g.user["user_id"], target_url),
    )

def pending_workspace_invites():
    return query_all(
        """
        SELECT wi.workspace_invitation_id, w.name AS workspace, u.username AS
invited_by,
        wi.invited_at
        FROM workspace_invitations AS wi
        JOIN workspaces AS w ON w.workspace_id = wi.workspace_id
        JOIN users AS u ON u.user_id = wi.invited_by
        WHERE wi.status = 'pending'
        AND (wi.invited_user_id = %s OR lower(wi.invited_email) = lower(%s))
        ORDER BY wi.invited_at
        """,
        (g.user["user_id"], g.user["email"]),
    )

```

```

def pending_channel_invites():
    return query_all(
        """
        SELECT ci.channel_invitation_id, c.name AS channel, w.name AS workspace,
               u.username AS invited_by, ci.invited_at
        FROM channel_invitations AS ci
        JOIN channels AS c ON c.channel_id = ci.channel_id
        JOIN workspaces AS w ON w.workspace_id = c.workspace_id
        JOIN users AS u ON u.user_id = ci.invited_by
        JOIN workspace_members AS wm
           ON wm.workspace_id = c.workspace_id AND wm.user_id = ci.invited_user_id
        WHERE ci.status = 'pending' AND ci.invited_user_id = %s
        ORDER BY ci.invited_at
        """,
        (g.user["user_id"],),
    )

def login_required(view):
    @wraps(view)
    def wrapped_view(**kwargs):
        if g.user is None:
            return redirect(url_for("login", next=request.path))
        return view(**kwargs)

    return wrapped_view

@app.before_request
def load_logged_in_user():
    user_id = session.get("user_id")
    g.user = None
    if user_id is not None:
        g.user = query_one(
            "SELECT user_id, email, username, nickname FROM users WHERE user_id = %s",
            (user_id,),
        )

def database_has_schema():
    row = query_one("SELECT to_regclass('public.users') AS users_table")
    return row and row["users_table"] is not None

@app.cli.command("init-db")
@click.option("--reset", is_flag=True, help="Drop and recreate existing tables.")
def init_db_command(reset):
    db = get_db()
    if database_has_schema() and not reset:
        click.echo(
            "Database already has snickr tables. Nothing changed. "
            "Run `flask --app app init-db --reset` if you want to rebuild sample data.
"
        )
    return

with db, db.cursor() as cur:
    for filename in ("schema.sql", "sample_data.sql"):
        with open(filename, encoding="utf-8") as f:

```

```

        cur.execute(f.read())
    click.echo("Initialized the snickr database with schema.sql and sample_data.sql.")

def verify_password(stored_hash, password):
    if stored_hash.startswith(("scrypt:", "pbkdf2:", "argon2:")):
        return check_password_hash(stored_hash, password)
    return stored_hash == password

@app.route("/")
def index():
    if g.user:
        return redirect(url_for("dashboard"))
    return redirect(url_for("login"))

@app.route("/register", methods=("GET", "POST"))
def register():
    if request.method == "POST":
        email = request.form["email"].strip().lower()
        username = request.form["username"].strip()
        nickname = request.form["nickname"].strip()
        password = request.form["password"]
        if not email or not username or not nickname or not password:
            flash("All fields are required.", "error")
        else:
            try:
                with get_db(), get_db().cursor() as cur:
                    cur.execute(
                        """
                        INSERT INTO users(email, username, nickname, password_hash)
                        VALUES (%s, %s, %s, %s)
                        RETURNING user_id
                        """,
                        (email, username, nickname, generate_password_hash(password)),
                    )
                user = cur.fetchone()
                session.clear()
                session["user_id"] = user["user_id"]
                flash("Your account is ready.", "success")
                return redirect(url_for("dashboard"))
            except psycopg2.IntegrityError:
                get_db().rollback()
                flash("That email or username is already in use.", "error")
    return render_template("register.html")

@app.route("/login", methods=("GET", "POST"))
def login():
    if request.method == "POST":
        username = request.form["username"].strip()
        password = request.form["password"]
        user = query_one(
            """
            SELECT user_id, username, nickname, password_hash
            FROM users
            WHERE username = %s OR email = %s
            """,

```



```

        (username, username.lower()),
    )
    if user and verify_password(user["password_hash"], password):
        session.clear()
        session["user_id"] = user["user_id"]
        return redirect(request.args.get("next") or url_for("dashboard"))
    flash("Invalid username/email or password.", "error")
    return render_template("login.html")

@app.route("/logout", methods=("POST",))
def logout():
    session.clear()
    return redirect(url_for("login"))

@app.route("/bookmarks", methods=("POST",))
@login_required
def add_bookmark():
    target = normalize_bookmark_target(request.form.get("target_url", ""))
    if target is None:
        flash("Only workspace, channel, and search pages can be bookmarked.", "error")
        return redirect(url_for("dashboard"))
    target_type, target_url = target
    label = request.form.get("label", "").strip() or target_url
    next_url = request.form.get("next", "").strip() or target_url
    ensure_bookmarks_schema()
    with get_db(), get_db().cursor() as cur:
        cur.execute(
            """
            INSERT INTO bookmarks(user_id, label, target_type, target_url)
            VALUES (%s, %s, %s, %s)
            ON CONFLICT (user_id, target_url) DO UPDATE
            SET label = EXCLUDED.label,
                target_type = EXCLUDED.target_type
            """
            ,
            (g.user["user_id"], label[:160], target_type, target_url),
        )
    flash("Bookmark saved.", "success")
    return redirect(next_url if normalize_bookmark_target(next_url) else url_for("dashboard"))

@app.route("/bookmarks/delete", methods=("POST",))
@login_required
def delete_bookmark():
    target = normalize_bookmark_target(request.form.get("target_url", ""))
    next_url = request.form.get("next", "").strip()
    if target is None:
        flash("Bookmark not found.", "error")
        return redirect(url_for("dashboard"))
    _target_type, target_url = target
    ensure_bookmarks_schema()
    with get_db(), get_db().cursor() as cur:
        cur.execute(
            "DELETE FROM bookmarks WHERE user_id = %s AND target_url = %s",
            (g.user["user_id"], target_url),
        )
    flash("Bookmark removed.", "success")

```

```

    return redirect(next_url if normalize_bookmark_target(next_url) else url_for("
dashboard"))

```

```

@app.route("/dashboard", methods=("GET", "POST"))

```

```

@login_required

```

```

def dashboard():

```

```

    if request.method == "POST":
        name = request.form["name"].strip()
        description = request.form["description"].strip()
        if not name:
            flash("Workspace name is required.", "error")
        else:
            try:
                with get_db(), get_db().cursor() as cur:
                    cur.execute(
                        """
                        INSERT INTO workspaces(name, description, created_by)
                        VALUES (%s, %s, %s)
                        RETURNING workspace_id
                        """,
                        (name, description, g.user["user_id"]),
                    )
                workspace = cur.fetchone()
                cur.execute(
                    """
                    INSERT INTO workspace_members(workspace_id, user_id, role)
                    VALUES (%s, %s, 'admin')
                    """,
                    (workspace["workspace_id"], g.user["user_id"]),
                )
                flash("Workspace created.", "success")
            except psycpg2.IntegrityError:
                get_db().rollback()
                flash("A workspace with that name already exists.", "error")
            return redirect(url_for("workspace", workspace_id=workspace["
workspace_id"]))

```

```

    except psycpg2.IntegrityError:
        get_db().rollback()
        flash("A workspace with that name already exists.", "error")

workspaces = query_all(
    """
    SELECT w.workspace_id, w.name, w.description, wm.role,
           COUNT(DISTINCT cm.channel_id) AS joined_channels
    FROM workspace_members AS wm
    JOIN workspaces AS w ON w.workspace_id = wm.workspace_id
    LEFT JOIN channels AS c ON c.workspace_id = w.workspace_id
    LEFT JOIN channel_members AS cm
        ON cm.channel_id = c.channel_id AND cm.user_id = wm.user_id
    WHERE wm.user_id = %s
    GROUP BY w.workspace_id, w.name, w.description, wm.role
    ORDER BY w.name
    """,
    (g.user["user_id"],),
)

```

```

workspace_invites = pending_workspace_invites()

```

```

channel_invites = pending_channel_invites()

```

```

ensure_bookmarks_schema()

```

```

bookmarks = query_all(

```

```

    """
    SELECT bookmark_id, label, target_type, target_url, created_at

```

```

        FROM bookmarks
        WHERE user_id = %s
        ORDER BY created_at DESC, bookmark_id DESC
        """
        (g.user["user_id"],),
    )
    return render_template(
        "dashboard.html",
        workspaces=workspaces,
        workspace_invites=workspace_invites,
        channel_invites=channel_invites,
        bookmarks=bookmarks,
    )

@app.route("/dashboard/invitations.json")
@login_required
def dashboard_invitations_json():
    workspace_invites = pending_workspace_invites()
    channel_invites = pending_channel_invites()
    return jsonify(
        {
            "workspace_invites": [
                {
                    "id": invite["workspace_invitation_id"],
                    "workspace": invite["workspace"],
                    "invited_by": invite["invited_by"],
                    "accept_url": url_for(
                        "respond_workspace_invitation",
                        invitation_id=invite["workspace_invitation_id"],
                        decision="accepted",
                    ),
                    "decline_url": url_for(
                        "respond_workspace_invitation",
                        invitation_id=invite["workspace_invitation_id"],
                        decision="declined",
                    ),
                }
                for invite in workspace_invites
            ],
            "channel_invites": [
                {
                    "id": invite["channel_invitation_id"],
                    "workspace": invite["workspace"],
                    "channel": invite["channel"],
                    "invited_by": invite["invited_by"],
                    "accept_url": url_for(
                        "respond_channel_invitation",
                        invitation_id=invite["channel_invitation_id"],
                        decision="accepted",
                    ),
                    "decline_url": url_for(
                        "respond_channel_invitation",
                        invitation_id=invite["channel_invitation_id"],
                        decision="declined",
                    ),
                }
                for invite in channel_invites
            ],
        }
    )

```

```

    }
)

def workspace_membership(workspace_id):
    return query_one(
        """
        SELECT role
        FROM workspace_members
        WHERE workspace_id = %s AND user_id = %s
        """,
        (workspace_id, g.user["user_id"]),
    )

def unique_usernames(usernames):
    seen = set()
    unique = []
    for username in usernames:
        if username not in seen:
            seen.add(username)
            unique.append(username)
    return unique

def workspace_invitee_rows(workspace_id, usernames):
    if not usernames:
        return []
    return query_all(
        """
        SELECT u.user_id, u.username
        FROM users AS u
        JOIN workspace_members AS wm ON wm.user_id = u.user_id
        WHERE wm.workspace_id = %s
        AND u.username = ANY(%s)
        """,
        (workspace_id, usernames),
    )

@app.route("/workspaces/<int:workspace_id>", methods=("GET", "POST"))
@login_required
def workspace(workspace_id):
    membership = workspace_membership(workspace_id)
    if membership is None:
        flash("You are not a member of that workspace.", "error")
        return redirect(url_for("dashboard"))

    workspace_row = query_one(
        "SELECT workspace_id, name, description FROM workspaces WHERE workspace_id = %s",
        (workspace_id,),
    )
    if request.method == "POST":
        channel_name = request.form["channel_name"].strip()
        channel_type = request.form["channel_type"]
        invitees = unique_usernames(
            [u.strip() for u in request.form.get("invitees", "").split(",") if u.strip()]
        )

```

```

)
if not channel_name:
    flash("Channel name is required.", "error")
elif channel_type not in {"public", "private", "direct"}:
    flash("Invalid channel type.", "error")
else:
    invitee_rows = workspace_invitee_rows(workspace_id, invitees)
    if invitees:
        found_usernames = {row["username"] for row in invitee_rows}
        invalid_invitees = [
            username for username in invitees if username not in
found_usernames
        ]
        if invalid_invitees:
            flash(
                "Cannot invite users who are not members of this workspace: "
                + ", ".join(invalid_invitees),
                "error",
            )
            return redirect(url_for("workspace", workspace_id=workspace_id))
    if channel_type == "direct":
        other_invitees = [
            row for row in invitee_rows if row["user_id"] != g.user["user_id"]
        ]
        if len(other_invitees) != 1:
            flash("Direct channels must include exactly one other workspace
member.", "error")
            return redirect(url_for("workspace", workspace_id=workspace_id))
    try:
        with get_db(), get_db().cursor() as cur:
            cur.execute(
                """
                INSERT INTO channels(workspace_id, name, channel_type,
created_by)
                VALUES (%s, %s, %s, %s)
                RETURNING channel_id
                """,
                (workspace_id, channel_name, channel_type, g.user["user_id"]),
            )
            channel = cur.fetchone()
            cur.execute(
                "INSERT INTO channel_members(channel_id, user_id) VALUES (%s,
%s)",
                (channel["channel_id"], g.user["user_id"]),
            )
            if channel_type == "direct":
                cur.execute(
                    """
                    INSERT INTO channel_members(channel_id, user_id)
                    VALUES (%s, %s)
                    ON CONFLICT (channel_id, user_id) DO NOTHING
                    """,
                    (channel["channel_id"], other_invitees[0]["user_id"]),
                )
            else:
                for invitee in invitee_rows:
                    if invitee["user_id"] == g.user["user_id"]:
                        continue
                    cur.execute(

```

```

        """
        INSERT INTO channel_invitations(
            channel_id, invited_user_id, invited_by
        )
        VALUES (%s, %s, %s)
        ON CONFLICT (channel_id, invited_user_id) DO NOTHING
        """
        (
            channel["channel_id"],
            invitee["user_id"],
            g.user["user_id"],
        ),
    )
    flash("Channel created.", "success")
    return redirect(url_for("channel", channel_id=channel["channel_id"]))
except psycopg2.IntegrityError:
    get_db().rollback()
    flash("That channel name already exists in this workspace.", "error")

channels = query_all(
    """
    SELECT c.channel_id, c.name, c.channel_type, c.created_at,
           (cm.user_id IS NOT NULL) AS is_member,
           COUNT(m.message_id) AS message_count
    FROM channels AS c
    LEFT JOIN channel_members AS cm
        ON cm.channel_id = c.channel_id AND cm.user_id = %s
    LEFT JOIN messages AS m ON m.channel_id = c.channel_id
    WHERE c.workspace_id = %s
        AND (c.channel_type = 'public' OR cm.user_id IS NOT NULL)
    GROUP BY c.channel_id, c.name, c.channel_type, c.created_at, cm.user_id
    ORDER BY c.channel_type, c.name
    """,
    (g.user["user_id"], workspace_id),
)

members = query_all(
    """
    SELECT u.username, u.nickname, wm.role
    FROM workspace_members AS wm
    JOIN users AS u ON u.user_id = wm.user_id
    WHERE wm.workspace_id = %s
    ORDER BY wm.role, u.username
    """,
    (workspace_id,),
)
return render_template(
    "workspace.html",
    workspace=workspace_row,
    membership=membership,
    channels=channels,
    members=members,
    bookmarked=bookmark_for(url_for("workspace", workspace_id=workspace_id)),
    bookmark_url=url_for("workspace", workspace_id=workspace_id),
)

@app.route("/workspaces/<int:workspace_id>/invite", methods=("POST",))
@login_required
def invite_workspace(workspace_id):

```

```

membership = workspace_membership(workspace_id)
if membership is None or membership["role"] != "admin":
    flash("Only workspace admins can invite workspace members.", "error")
    return redirect(url_for("workspace", workspace_id=workspace_id))
invitee_text = request.form["invitee"].strip()
if not invitee_text:
    flash("Enter an email address or username.", "error")
    return redirect(url_for("workspace", workspace_id=workspace_id))
if "@" in invitee_text:
    invitee = query_one(
        "SELECT user_id, email FROM users WHERE lower(email) = lower(%s)",
        (invitee_text.lower(),),
    )
    invited_email = invitee["email"] if invitee else invitee_text.lower()
else:
    invitee = query_one(
        "SELECT user_id, email FROM users WHERE username = %s",
        (invitee_text,),
    )
    if invitee is None:
        flash("No user with that username exists. Use an email address to invite a new user.", "error")
        return redirect(url_for("workspace", workspace_id=workspace_id))
    invited_email = invitee["email"]
try:
    with get_db(), get_db().cursor() as cur:
        cur.execute(
            """
            INSERT INTO workspace_invitations(
                workspace_id, invited_email, invited_user_id, invited_by
            )
            VALUES (%s, %s, %s, %s)
            ON CONFLICT (workspace_id, invited_email) DO NOTHING
            """,
            (
                workspace_id,
                invited_email,
                invitee["user_id"] if invitee else None,
                g.user["user_id"],
            ),
        )
        flash("Workspace invitation recorded.", "success")
except psycopg2.Error:
    get_db().rollback()
    flash("Could not create that invitation.", "error")
return redirect(url_for("workspace", workspace_id=workspace_id))

@app.route("/workspace-invitations/<int:invitation_id>/<decision>", methods=("POST",))
@login_required
def respond_workspace_invitation(invitation_id, decision):
    if decision not in {"accepted", "declined"}:
        flash("Invalid invitation response.", "error")
        return redirect(url_for("dashboard"))
    invitation = query_one(
        """
        SELECT workspace_invitation_id, workspace_id
        FROM workspace_invitations
        WHERE workspace_invitation_id = %s
        """
    )

```

```

        AND status = 'pending'
        AND (invited_user_id = %s OR lower(invited_email) = lower(%s))
    """
    (invitation_id, g.user["user_id"], g.user["email"]),
)
if invitation is None:
    flash("Invitation not found.", "error")
    return redirect(url_for("dashboard"))
with get_db(), get_db().cursor() as cur:
    cur.execute(
        """
        UPDATE workspace_invitations
        SET status = %s, responded_at = CURRENT_TIMESTAMP
        WHERE workspace_invitation_id = %s
        """,
        (decision, invitation_id),
    )
    if decision == "accepted":
        cur.execute(
            """
            INSERT INTO workspace_members(workspace_id, user_id, role)
            VALUES (%s, %s, 'member')
            ON CONFLICT (workspace_id, user_id) DO NOTHING
            """,
            (invitation["workspace_id"], g.user["user_id"]),
        )
flash("Invitation response saved.", "success")
return redirect(url_for("dashboard"))

@app.route("/channels/<int:channel_id>", methods=("GET", "POST"))
@login_required
def channel(channel_id):
    ensure_message_withdrawal_schema()
    channel_row = query_one(
        """
        SELECT c.channel_id, c.name, c.channel_type, c.workspace_id, w.name AS
workspace,
        (cm.user_id IS NOT NULL) AS is_member
        FROM channels AS c
        JOIN workspaces AS w ON w.workspace_id = c.workspace_id
        JOIN workspace_members AS wm
        ON wm.workspace_id = c.workspace_id AND wm.user_id = %s
        LEFT JOIN channel_members AS cm
        ON cm.channel_id = c.channel_id AND cm.user_id = %s
        WHERE c.channel_id = %s
        AND (c.channel_type = 'public' OR cm.user_id IS NOT NULL)
        """,
        (g.user["user_id"], g.user["user_id"], channel_id),
    )
    if channel_row is None:
        flash("You are not a member of that channel.", "error")
        return redirect(url_for("dashboard"))
    if channel_row["channel_type"] == "public" and not channel_row["is_member"]:
        with get_db(), get_db().cursor() as cur:
            cur.execute(
                """
                INSERT INTO channel_members(channel_id, user_id)
                VALUES (%s, %s)
            """

```



```

        ON CONFLICT (channel_id, user_id) DO NOTHING
        """
        (channel_id, g.user["user_id"]),
    )
    channel_row["is_member"] = True
if request.method == "POST":
    body = request.form["body"].strip()
    if not body:
        flash("Message body cannot be blank.", "error")
    else:
        with get_db(), get_db().cursor() as cur:
            cur.execute(
                """
                INSERT INTO messages(channel_id, sender_id, body)
                VALUES (%s, %s, %s)
                """,
                (channel_id, g.user["user_id"], body),
            )
            flash("Message posted.", "success")
            return redirect(url_for("channel", channel_id=channel_id))
messages = query_all(
    """
    SELECT m.message_id, m.body, m.posted_at, m.sender_id, m.withdrawn_at,
           u.username, u.nickname
    FROM messages AS m
    JOIN users AS u ON u.user_id = m.sender_id
    WHERE m.channel_id = %s
    ORDER BY m.posted_at, m.message_id
    """,
    (channel_id,),
)
message_urls = [
    url_for("channel", channel_id=channel_id) + f"#message-{message['message_id']}"
    for message in messages
]
ensure_bookmarks_schema()
message_bookmarks = set()
if message_urls:
    message_bookmarks = {
        row["target_url"]
        for row in query_all(
            """
            SELECT target_url
            FROM bookmarks
            WHERE user_id = %s
            AND target_url = ANY(%s)
            """,
            (g.user["user_id"], message_urls),
        )
    }
members = query_all(
    """
    SELECT u.username, u.nickname
    FROM channel_members AS cm
    JOIN users AS u ON u.user_id = cm.user_id
    WHERE cm.channel_id = %s
    ORDER BY u.username
    """,

```

```

        (channel_id,),
    )
    return render_template(
        "channel.html",
        channel=channel_row,
        messages=messages,
        members=members,
        bookmarked=bookmark_for(url_for("channel", channel_id=channel_id)),
        bookmark_url=url_for("channel", channel_id=channel_id),
        message_bookmarks=message_bookmarks,
    )

@app.route("/channels/<int:channel_id>/messages.json")
@login_required
def channel_messages_json(channel_id):
    ensure_message_withdrawal_schema()
    try:
        after = int(request.args.get("after", 0))
    except ValueError:
        after = 0
    channel_row = query_one(
        """
        SELECT c.channel_id, c.channel_type
        FROM channels AS c
        JOIN workspace_members AS wm
          ON wm.workspace_id = c.workspace_id AND wm.user_id = %s
        LEFT JOIN channel_members AS cm
          ON cm.channel_id = c.channel_id AND cm.user_id = %s
        WHERE c.channel_id = %s
          AND (c.channel_type = 'public' OR cm.user_id IS NOT NULL)
        """,
        (g.user["user_id"], g.user["user_id"], channel_id),
    )
    if channel_row is None:
        return jsonify({"messages": []}), 403
    messages = query_all(
        """
        SELECT m.message_id, m.body, m.posted_at, m.sender_id, m.withdrawn_at,
               u.username, u.nickname
        FROM messages AS m
        JOIN users AS u ON u.user_id = m.sender_id
        WHERE m.channel_id = %s
          AND m.message_id > %s
        ORDER BY m.posted_at, m.message_id
        LIMIT 100
        """,
        (channel_id, after),
    )
    withdrawals = query_all(
        """
        SELECT message_id
        FROM messages
        WHERE channel_id = %s
          AND withdrawn_at IS NOT NULL
        """,
        (channel_id,),
    )
    return jsonify(

```

```

    {
        "messages": [
            {
                "message_id": message["message_id"],
                "body": message["body"],
                "posted_at": message["posted_at"].strftime("%Y-%m-%d %H:%M"),
                "sender_id": message["sender_id"],
                "username": message["username"],
                "nickname": message["nickname"],
                "withdrawn": message["withdrawn_at"] is not None,
            }
            for message in messages
        ],
        "withdrawals": [message["message_id"] for message in withdrawals],
    }
)

@app.route("/messages/<int:message_id>/withdraw", methods=("POST",))
@login_required
def withdraw_message(message_id):
    ensure_message_withdrawal_schema()
    message = query_one(
        """
        SELECT m.message_id, m.sender_id, m.channel_id, m.withdrawn_at,
               c.workspace_id, c.channel_type
        FROM messages AS m
        JOIN channels AS c ON c.channel_id = m.channel_id
        JOIN workspace_members AS wm
            ON wm.workspace_id = c.workspace_id AND wm.user_id = %s
        LEFT JOIN channel_members AS cm
            ON cm.channel_id = c.channel_id AND cm.user_id = %s
        WHERE m.message_id = %s
            AND (c.channel_type = 'public' OR cm.user_id IS NOT NULL)
        """,
        (g.user["user_id"], g.user["user_id"], message_id),
    )
    if message is None:
        flash("Message not found.", "error")
        return redirect(url_for("dashboard"))
    if message["sender_id"] != g.user["user_id"]:
        flash("You can only withdraw your own messages.", "error")
        return redirect(url_for("channel", channel_id=message["channel_id"]))
    if message["withdrawn_at"] is None:
        message_bookmark_url = (
            url_for("channel", channel_id=message["channel_id"])
            + f"#message-{message_id}"
        )
        ensure_bookmarks_schema()
        with get_db(), get_db().cursor() as cur:
            cur.execute(
                """
                UPDATE messages
                SET withdrawn_at = CURRENT_TIMESTAMP,
                    withdrawn_by = %s
                WHERE message_id = %s
                    AND sender_id = %s
                    AND withdrawn_at IS NULL
                """,

```

```

        (g.user["user_id"], message_id, g.user["user_id"]),
    )
    cur.execute(
        """
        DELETE FROM bookmarks
        WHERE target_type = 'message'
          AND target_url = %s
        """,
        (message_bookmark_url,),
    )
    flash("Message withdrawn.", "success")
else:
    flash("That message was already withdrawn.", "error")
return redirect(url_for("channel", channel_id=message["channel_id"]))

@app.route("/channels/<int:channel_id>/invite", methods=("POST",))
@login_required
def invite_channel(channel_id):
    channel_row = query_one(
        """
        SELECT c.channel_id, c.workspace_id, c.channel_type
        FROM channels AS c
        JOIN channel_members AS cm
          ON cm.channel_id = c.channel_id AND cm.user_id = %s
        WHERE c.channel_id = %s
        """,
        (g.user["user_id"], channel_id),
    )
    if channel_row is None:
        flash("You must belong to a channel before inviting others.", "error")
        return redirect(url_for("dashboard"))
    if channel_row["channel_type"] == "direct":
        flash("Direct channels are limited to their original participants.", "error")
        return redirect(url_for("channel", channel_id=channel_id))
    username = request.form["username"].strip()
    invitee = query_one(
        """
        SELECT u.user_id
        FROM users AS u
        JOIN workspace_members AS wm ON wm.user_id = u.user_id
        WHERE u.username = %s AND wm.workspace_id = %s
        """,
        (username, channel_row["workspace_id"]),
    )
    if invitee is None:
        flash("That user is not a member of this workspace.", "error")
    else:
        with get_db(), get_db().cursor() as cur:
            cur.execute(
                """
                INSERT INTO channel_invitations(channel_id, invited_user_id,
invited_by)
                VALUES (%s, %s, %s)
                ON CONFLICT (channel_id, invited_user_id) DO NOTHING
                """,
                (channel_id, invitee["user_id"], g.user["user_id"]),
            )
        flash("Channel invitation recorded.", "success")

```

```

        return redirect(url_for("channel", channel_id=channel_id))

@app.route("/channel-invitations/<int:invitation_id>/<decision>", methods=("POST",))
@login_required
def respond_channel_invitation(invitation_id, decision):
    if decision not in {"accepted", "declined"}:
        flash("Invalid invitation response.", "error")
        return redirect(url_for("dashboard"))
    invitation = query_one(
        """
        SELECT ci.channel_invitation_id, ci.channel_id, c.workspace_id
        FROM channel_invitations AS ci
        JOIN channels AS c ON c.channel_id = ci.channel_id
        WHERE ci.channel_invitation_id = %s
              AND ci.status = 'pending'
              AND ci.invited_user_id = %s
        """,
        (invitation_id, g.user["user_id"]),
    )
    if invitation is None:
        flash("Invitation not found.", "error")
        return redirect(url_for("dashboard"))
    if decision == "accepted" and workspace_membership(invitation["workspace_id"]) is None:
        flash("Join the workspace before accepting that channel invitation.", "error")
        return redirect(url_for("dashboard"))
    with get_db(), get_db().cursor() as cur:
        cur.execute(
            """
            UPDATE channel_invitations
            SET status = %s, responded_at = CURRENT_TIMESTAMP
            WHERE channel_invitation_id = %s
            """,
            (decision, invitation_id),
        )
        if decision == "accepted":
            cur.execute(
                """
                INSERT INTO channel_members(channel_id, user_id)
                VALUES (%s, %s)
                ON CONFLICT (channel_id, user_id) DO NOTHING
                """,
                (invitation["channel_id"], g.user["user_id"]),
            )
    flash("Invitation response saved.", "success")
    return redirect(url_for("dashboard"))

@app.route("/search")
@login_required
def search():
    ensure_message_withdrawal_schema()
    keyword = request.args.get("q", "").strip()
    results = []
    if keyword:
        results = query_all(
            """
            SELECT m.message_id, m.body, m.posted_at, u.username,

```

```

        c.channel_id, c.name AS channel, w.name AS workspace
FROM messages AS m
JOIN channels AS c ON c.channel_id = m.channel_id
JOIN workspaces AS w ON w.workspace_id = c.workspace_id
JOIN users AS u ON u.user_id = m.sender_id
JOIN workspace_members AS wm
    ON wm.workspace_id = w.workspace_id AND wm.user_id = %s
LEFT JOIN channel_members AS cm
    ON cm.channel_id = c.channel_id AND cm.user_id = %s
WHERE m.body ILIKE '%%' || %s || '%%'
    AND (c.channel_type = 'public' OR cm.user_id IS NOT NULL)
    AND m.withdrawn_at IS NULL
ORDER BY m.posted_at DESC, m.message_id DESC
""",
    (g.user["user_id"], g.user["user_id"], keyword),
)
bookmark_url = url_for("search", q=keyword) if keyword else None
return render_template(
    "search.html",
    keyword=keyword,
    results=results,
    bookmarked=bookmark_for(bookmark_url) if bookmark_url else None,
    bookmark_url=bookmark_url,
)

if __name__ == "__main__":
    app.run(debug=True)

```